

Contents

Nomenclature	iii
Acronyms	iv
1 Memory Machine: build a State Space Model from scratch	1
1.1 Let's begin with an Alien Invasion	1
1.2 Build a state space model from scratch	2
1.3 Mamba: when selectivity comes in	2
1.4 Walk through the Parameter Pipeline	3
2 HiPPO: where the state matrix comes from	4
2.1 The Illusion of the Perfect Formula	4
2.2 Deriving the HiPPO Matrices: Scaling the Window	4
2.2.1 The Measure	5
2.2.2 The Basis	5
2.2.3 The Coefficient Dynamics	5
2.3 Why Does This Matter?	5
3 Discretisation: turning continuous SSMs into discrete case	6
3.1 Numerical Integration	7
3.1.1 Forward Euler	7
3.1.2 Backward Euler	7
3.1.3 Bilinear / Trapezoidal	8
3.2 Geometric Derivation via Gradient Approximation	8
3.2.1 Forward Euler (Left-Sided Slope)	9
3.2.2 Backward Euler (Right-Sided Slope)	9
3.2.3 Bilinear / Trapezoidal (Average Slope)	9
3.3 Comparative Analysis: Selecting the Optimal Discretisation	10
3.3.1 Empirical Validation: The Reconstruction Task	10
3.4 The Exact Solution (ZOH)	12
3.5 Comparisons via Taylor Expansion	12
3.5.1 Summary of Discretisation Matrices	12
3.5.2 Order of Error Analysis	12
3.5.3 Conclusion	13
4 When God rolls the dice: ODE / RNN / CNN views	14
5 Why Linear Time-Invariant (LTI) is not enough	17
5.1 The Neuroscience of Memory Budgeting	17
5.2 The Information Theory of Surprisal	17
5.3 The Need for Selectivity	18
6 Selective State Space: how Mamba makes B, C, and Δ_t time-varying	19

7	The Evolution of the SSM State Matrix A	21
7.1	The general SSM form	21
7.2	How A evolve	21
7.3	HiPPO: A as an optimal history compressor	22
7.4	LSSL: HiPPO A + Bilinear discretisation	22
7.5	S4: Making A computational friendly with DPLR	23
7.5.1	From S4 to selective models: LTI becomes the bottleneck	24
7.6	Mamba-01: Time dependent diagonal A + ZOH discretisation	24
7.7	Mamba-02: Scalar Identity A as SSD	25
8	Duality: why a SSM can look like a causal Transformer	27
8.1	The Elephant in the Room: The Transformer	27
8.2	The Mathematical Bridge: Mamba-2 and the Scalar Identity	27
8.3	Duality	28
9	Mamba in engineering	29

Nomenclature

x_t Input at step t (token / observation)

h_t Hidden state (memory) at step t

y_t Output at step t

A State transition / state matrix

B Input (write) matrix

C Readout (read) matrix

D Skip / direct term from input to output

Δ_t Time step / discretization step size at step t

$\overline{\mathbf{A}}$ Discretized transition matrix

$\overline{\mathbf{B}}$ Discretized input matrix

$\overline{\mathbf{K}}$ Discrete convolution kernel of an SSM

$\mathbb{R}$ Set of real numbers

$\mathbb{C}$ Set of complex numbers

Acronyms

CNN Convolutional Neural Network

DPLR Diagonal Plus Low-Rank

FFT Fast Fourier Transform

LSSL Linear State Space Layer

LTl Linear Time-Invariant

ODE Ordinary Differential Equation

RAG Retrieval-Augmented Generation

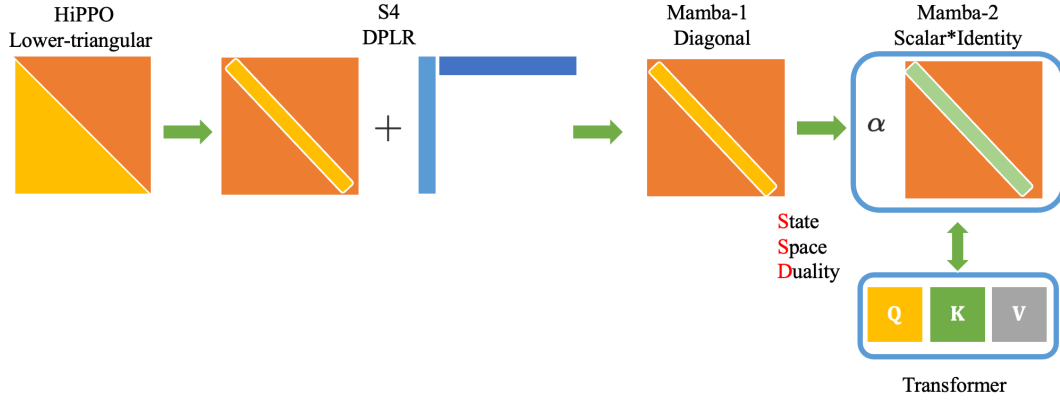
RNN Recurrent Neural Network

S4 Structured State Space Sequence Model

SSD State Space Duality

SSM State Space Model

ZOH Zero-Order Hold



1 Memory Machine: build a State Space Model from scratch

If you search for articles/blogs explaining Mamba, nine times out of ten, you will find an introduction that begins: *To address the limitations of the Transformer...*

This is a missed opportunity. Framing Mamba merely as a substitute or a patch for the Transformer obscures its independent elegance. For our story, let's briefly set the Transformer aside. We are going to start from scratch and focus on one single question: How do we build a machine that truly understands memory?

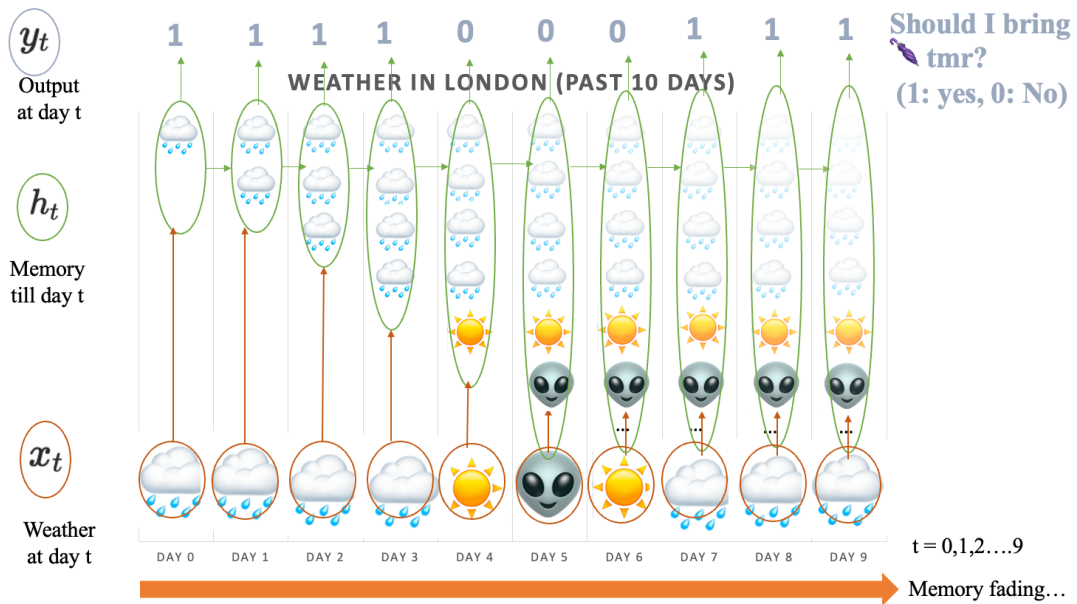


Figure 1

1.1 Let's begin with an Alien Invasion

To understand memory, let's look at a simple everyday task: remembering the weather in London over the past 10 days to decide whether to bring an umbrella tomorrow.

Our machine relies on three core components:

- x_t : The new input (e.g., today's weather).

- h_t : The hidden states (accumulated memory of past weather).
- y_t : The final action (1 for bringing an umbrella, 0 for leaving it).

London is notoriously rainy. Imagine it has rained for the past four days; your memory h_4 correctly tells you, “It is damp out there.” But on day 5, something utterly absurd happens—an **alien invasion** (this is our new input x_5).

At this moment, your “brain” (the memory state h_5 at $t = 5$) needs to merge “days of rain” with “alien arrival.” Intuitively, human memory fades over time. Older events become blurry.

The critical question is: When faced with new, wildly varying daily events, how fast should the machine forget the past? And how aggressively should it record the present?

1.2 Build a state space model from scratch

In basic mathematical models, we assume the blending of new and old memories is “linear,” functioning like three fixed dials on a control panel:

- **Dial A (Retention)**: Decides how much of yesterday’s memory h_{t-1} to keep.
- **Dial B (Absorption)**: Decides how much of today’s new information x_t to write down.
- **Dial C (Output)**: Decides how to translate the memory into the final umbrella decision y_t .

Expressed simply:

$$h_t = A h_{t-1} + B x_t, \tag{1}$$

$$y_t = C h_t. \tag{2}$$

In a traditional Linear Time-Invariant (LTI) model, these three dials are **welded in place** at the factory. This creates a fatal flaw: the machine is rigidly inflexible.

Neuroscience shows that human memory is highly elastic. During days of monotonous drizzle, our brains use a “**weak write + high forget**” mode. Why memorise every identical raindrop? But if aliens descend today, the massive stimulus triggers a “**strong write + low forget**” mode. You might entirely forget the week’s weather because your brain is completely consumed by spaceships.

A rigid LTI machine uses the exact same memory strength for both a light drizzle and an alien invasion. This defies logic. We need an adaptable machine—one where the parameters A , B , and C dynamically adjust based on what actually happens each day.

1.3 Mamba: when selectivity comes in

This is the core brilliance of Mamba. It gives the machine the ability to dynamically adjust its own dials.

Mamba approaches this by first analysing a continuous flow of time, then slicing it into discrete digital steps using a technique called Zero-Order Hold (ZOH). This introduces a crucial magic variable: **the time step** Δ_t (often written as dt_t).

Once we introduce Δ_t , our static dials A and B transform into dynamic, time-varying parameters $\bar{A}$ and $\bar{B}$. You can think of Δ_t as the machine’s “**attention span**” for a specific event:

- **When Δ_t is small**: The event is trivial (another boring rainy day). The machine increases $\bar{A}$ (holds onto the old memory) and decreases $\bar{B}$ (barely writes the new rain into memory).

- **When Δ_t is large:** A monumental event has occurred (Aliens!). The machine decreases $\bar{A}$ (clears out useless past memory) and increases $\bar{B}$ (burns the alien information into memory).

Our ultimate goal is now clear: **We must design a pipeline so that every time the machine looks at today’s input x_t , it automatically calculates today’s step size Δ_t , alongside the new read/write parameters B_t and C_t .**

1.4 Walk through the Parameter Pipeline

How does this automated tuning pipeline actually work? In Mamba’s architecture, it is elegantly broken down into four steps:

Step 1: Expanding the Canvas (in_proj). The raw weather record x_t might be too basic. To uncover deeper details, we first “magnify” it using a projection function, mapping it into a high-dimensional space to create a detail-rich vector u_t .

Step 2: Checking the Neighbourhood (1D Convolution). High-definition details of *today* aren’t enough. If we only know aliens arrived today, but lack yesterday’s context, we can’t tell if an alien invasion is a rare anomaly or a daily nuisance. Mamba uses a 1D convolution to blend u_t with the local context of the past few days, resulting in a smarter, context-aware $\tilde{u}_t$.

Step 3: Extracting the Dial Instructions (x_proj). Next, we take the context-rich $\tilde{u}_t$ and project it again, slicing it into three distinct pieces like a cake. These three pieces are our highly desired control instructions: the write parameter B_t , the read parameter C_t , and the raw blueprint for our time step, Δ_{raw} .

Step 4: Fine-Tuning the Time Step (dt_proj and Log-uniform Sampling). You might ask: *If we have Δ_{raw} , aren’t we done?*

Not quite. For computational efficiency, Δ_{raw} was heavily compressed. We need to project it back to its normal dimension and apply a **softplus** activation function to ensure the time step is always a positive number.

There is also a brilliant engineering trick hidden here regarding **bias initialisation**.

Suppose we want our step size Δ to comfortably fall between 10^{-3} and 10^{-1} . If we use standard “uniform” sampling to initialize this bias, most values will cluster near 10^{-1} (since the numerical distance between 0.01 and 0.1 is much larger than between 0.001 and 0.01). This would make the machine naturally biased toward “overreacting.”

To fix this, Mamba uses **log-uniform sampling** to initialise the bias. This ensures the machine maintains an objective, balanced sensitivity across entirely different magnitudes of stimuli.

Armed with the custom-generated Δ_t , B_t , and C_t for that specific day, Mamba flawlessly executes its dynamic, context-aware memory updates.

2 HiPPO: where the state matrix comes from

Before we dive into the dense mathematics of matrices, let’s take a detour into the human brain.

In cognitive neuroscience, human memory is fundamentally understood to be *reconstructive*. Our brains do not work like computer hard drives, storing raw video files of our past. Instead, the hippocampal-medial temporal lobe acts more like an index. It uses partial cues to reactivate distributed cortical traces, allowing us to vividly rebuild past episodes based on the demands of the current moment. This reconstructive mechanism allows for flexible recall without the biological impossibility of passive, exhaustive storage.

If we view modern Large Language Models through this lens, architectures using Retrieval-Augmented Generation (RAG) act much like an “external brain,” fetching explicitly stored external memories. Mamba, however, hints at something deeper: a sort of *native retrieval*. It computationally formalizes this neuroscientific principle. It compresses extensive historical data into a finite-dimensional state, yet stubbornly maintains the ability to reconstruct the original signal from that compressed state.

But how exactly do we reconstruct a stream of data mathematically?

2.1 The Illusion of the Perfect Formula

In junior high math classes, we are taught that if we can explicitly write down a function’s expression—say, $f(t) = t^2 + 3t$ —we have solved the problem. An explicit analytical function is the ultimate, perfect form of compression; it represents an infinite number of data points with just a few symbols.

But the real world is rarely so accommodating. In most deep learning tasks, we do not have an explicit generating formula for our data, nor can we assume the data originates from a simple closed-form expression.

So, we naturally settle for the next best thing: approximation. If we do not know the true $f(t)$, can we use a set of familiar, well-behaved functions to approximate it?

Imagine selecting a set of well-structured basis functions, $(\phi_1(t), \phi_2(t), \dots, \phi_N(t))$. We can approximate our messy, real-world data using a linear combination of these N basis functions:

$$f(t) \approx \sum_{n=1}^N c_n \phi_n(t). \quad (3)$$

Suddenly, the objective of our memory machine shifts. The reconstruction problem transforms from the impossible task of “finding an unknown function” to the highly solvable task of “finding a vector of coefficients” $c = (c_1, \dots, c_N)$. Once we possess the coefficient vector c , we can not only accurately reconstruct the data we have already observed, but we can also perform logical reasoning on unobserved points.

In mathematical terms, this narrative perfectly aligns with the framework of a Hilbert space. The target data $f(t)$ belongs to a vast function space, and our chosen basis functions expand to form an N -dimensional subspace. Approximation is simply the act of projecting $f(t)$ into this subspace to find the optimal shadow. Given this function approximation formulation, orthogonal polynomials emerge as a natural choice for our basis functions because their optimal coefficients can be gracefully calculated in closed form.

2.2 Deriving the HiPPO Matrices: Scaling the Window

The HiPPO (High-order Polynomial Projection Operators) framework is the mathematical engine that makes this possible. Let’s look under the hood at how the famous HiPPO matrices A and B are actually derived from first principles, specifically focusing on the most robust variant: the Scaled Legendre measure (HiPPO-LegS).

To project our history, we need two ingredients: a *measure* (how much we care about different parts of the past) and a *basis* (the functions we use to approximate).

2.2.1 The Measure

Unlike a sliding window that strictly forgets everything outside a narrow timeframe, the Scaled Legendre measure (LegS) uniformly weighs the *entire* history from time 0 to the current time t . Mathematically, this time-varying weight is defined as:

$$\omega(t, x) = \frac{1}{t} \mathbb{I}_{[0, t]}. \quad (4)$$

2.2.2 The Basis

For our basis, we utilize Legendre polynomials. Because our window scales dynamically from 0 to t , we must shift and scale the standard Legendre polynomials to fit perfectly within this moving window:

$$g_n(t, x) = (2n + 1)^{1/2} P_n\left(\frac{2x}{t} - 1\right). \quad (5)$$

2.2.3 The Coefficient Dynamics

Here is where the magic happens. Remember our goal: we want to track the coefficients $c(t)$ over time. If we differentiate the projection equation with respect to time t , we are asking the system: *“How should my coefficients change right now to optimally incorporate this new data point while maintaining my memory of the past?”*

After carefully differentiating the measure and the basis polynomials and leveraging the recursive properties of Legendre polynomials, the complex integrals elegantly collapse into a beautiful, continuous linear ordinary differential equation (ODE):

$$\frac{d}{dt} c(t) = -\frac{1}{t} A c(t) + \frac{1}{t} B f(t). \quad (6)$$

Where the matrices A and B drop out as mathematically exact constants:

$$A_{nk} = \begin{cases} (2n + 1)^{1/2} (2k + 1)^{1/2} & \text{if } n > k, \\ n + 1 & \text{if } n = k, \\ 0 & \text{if } n < k, \end{cases} \quad (7)$$

$$B_n = (2n + 1)^{1/2}. \quad (8)$$

With these mathematically derived dynamics, at any given time t , the system can flawlessly reconstruct its history using the coefficients it has tracked:

$$f(x) \approx g^{(t)}(x) = \sum_n c_n(t) (2n + 1)^{1/2} P_n\left(\frac{2x}{t} - 1\right). \quad (9)$$

2.3 Why Does This Matter?

The State Matrix A in Mamba is not just a random grid of numbers that a neural network blindly learns through brute-force gradient descent. It is deeply rooted in approximation theory. The HiPPO framework guarantees that as your sequence length approaches millions of tokens, the A matrix is mathematically optimal for compressing that history into a finite state space. It is a memory system that does not just store data; it understands how to reconstruct it.

3 Discretisation: turning continuous SSMs into discrete case

The fundamental innovation of modern State Space Models (such as LSSL and S4) lies in their ability to be interpreted through three distinct, yet mathematically equivalent, lenses. As shown in Figure 2, each view offers specific computational or theoretical advantages:

1. **The Continuous View (ODE):**

$$\dot{x}(t) = Ax(t) + Bu(t) \quad (10)$$

The continuous view forms the theoretical foundation. It aligns with the continuous nature of real-world signals, enabling the use of HiPPO matrices to model long-range memory.

2. **The Recurrent View (RNN):**

$$x_{k+1} = \bar{A}x_k + \bar{B}u_k \quad (11)$$

This implies a step-by-step evolution of the state. It is essential for efficient **inference** where inputs arrive sequentially.

3. **The Convolutional View (CNN):**

$$y = \bar{K} * u \quad (12)$$

This views the system as a filter. It is critical for efficient **training**, allowing parallel computation over the entire sequence.

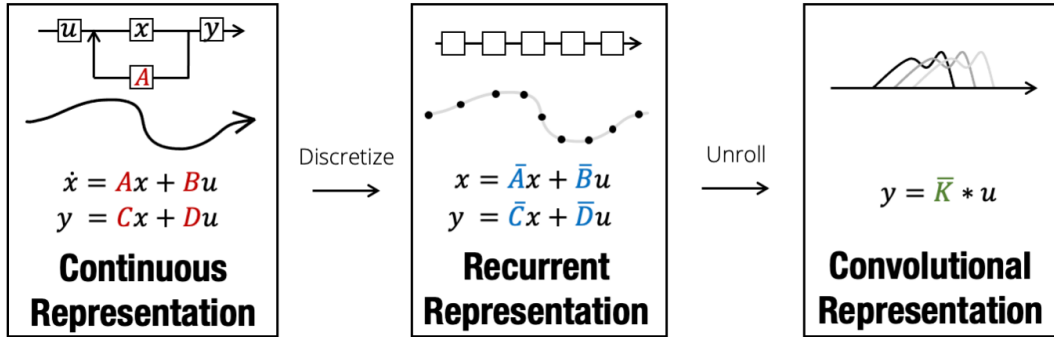


Figure 2: **The SSM Representations.** The model can be transformed from a continuous ODE (left) to a discrete recurrence (centre) and unrolled into a global convolution (right).[?]

To transfer from the theoretical **Continuous View** to **Recurrent View**, we must transform the continuous parameters (A, B) into discrete parameters $(\bar{A}, \bar{B})$ using a time step Δt . This process is termed **Discretisation**.

There is no single "correct" method to perform this transformation. In this appendix, we derive the discrete matrices by analysing the problem from three different methodological perspectives:

- **Perspective 1: Numerical Integration.** We approximate the area under the derivative curve (solving $\int \dot{x} d\tau$).
- **Perspective 2: Gradient Approximation.** We project the state forward using geometric slope approximations (Forward, Backward, and Average slopes).

- **Perspective 3: The Exact Solution (ZOH).** We derive the analytical solution of the ODE assuming a Zero-Order Hold on the input.

The following sections detail these derivations and theoretically justify why the **Bilinear (Trapezoidal)** method is the preferred choice for models like the LSSL via both empirical validation and comparisons on Taylor series expansion.

3.1 Numerical Integration

We start with the continuous-time linear time-invariant (LTI) ordinary differential equation (ODE):

$$\dot{x}(t) = Ax(t) + Bu(t) \quad (13)$$

To determine the state evolution over the interval $[t, t + \Delta t]$, we integrate both sides of the equation:

$$\int_t^{t+\Delta t} \dot{x}(\tau) d\tau = \int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau)) d\tau \quad (14)$$

Evaluating the left-hand side yields the difference between the final and initial states. For notational brevity, we define $x_k = x(t)$ and $x_{k+1} = x(t + \Delta t)$:

$$x_{k+1} - x_k = \int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau)) d\tau \quad (15)$$

Following the integration in Eq. (15), the fundamental challenge lies in evaluating the definite integral on the right-hand side. As the trajectories of the state $x(\tau)$ and input $u(\tau)$ are unknown within the interval $\tau \in [t, t + \Delta t]$, we must employ numerical approximation methods.

We can approximate the integral $\int_t^{t+\Delta t} f(\tau) d\tau \approx \Delta t \cdot \Phi$ using different schemes for the slope Φ . This results in three distinct discretisation methods: Forward Euler, Backward Euler, and the Bilinear (Trapezoidal) transformation.

3.1.1 Forward Euler

The Forward Euler method approximates the area under the curve using a rectangle based on the gradient at the *start* of the interval (time t). We assume the derivatives remain constant at their initial values:

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau)) d\tau \approx \Delta t (Ax_k + Bu_k) \quad (16)$$

$$\begin{aligned} \implies x_{k+1} - x_k &= \Delta t Ax_k + \Delta t Bu_k \\ \implies x_{k+1} &= (I + \Delta t A)x_k + \Delta t Bu_k \end{aligned} \quad (17)$$

Note: As shown in Eq. (17), this method requires no matrix inversion but is only conditionally stable.

3.1.2 Backward Euler

The Backward Euler method uses the gradient at the *end* of the interval ($t + \Delta t$). We assume values at the next time step dominate:

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau)) d\tau \approx \Delta t (Ax_{k+1} + Bu_{k+1}) \quad (18)$$

Substituting this yields an implicit equation:

$$\begin{aligned}
x_{k+1} - x_k &= \Delta t A x_{k+1} + \Delta t B u_{k+1} \\
(I - \Delta t A) x_{k+1} &= x_k + \Delta t B u_{k+1} \\
x_{k+1} &= (I - \Delta t A)^{-1} x_k + (I - \Delta t A)^{-1} \Delta t B u_{k+1}
\end{aligned} \tag{19}$$

Eq. (19) shows that this method requires matrix inversion.

3.1.3 Bilinear / Trapezoidal

The Bilinear transform (Tustin's method) approximates the integral using the area of a trapezoid, effectively averaging the gradients at the start and end of the interval. This offers a balance between the Forward and Backward schemes and preserves the stability boundary of the continuous system.

$$\int_t^{t+\Delta t} (Ax(\tau) + Bu(\tau)) d\tau \approx \frac{\Delta t}{2} (Ax_k + Ax_{k+1}) + \frac{\Delta t}{2} (Bu_k + Bu_{k+1}) \tag{20}$$

Substituting this into the state equation:

$$x_{k+1} - x_k = \frac{\Delta t}{2} A(x_k + x_{k+1}) + \frac{\Delta t}{2} B(u_k + u_{k+1}) \tag{21}$$

To isolate x_{k+1} , we collect terms on the left-hand side:

$$\left(I - \frac{\Delta t}{2} A\right) x_{k+1} = \left(I + \frac{\Delta t}{2} A\right) x_k + \frac{\Delta t}{2} B(u_k + u_{k+1}) \tag{22}$$

Finally, applying the matrix inverse yields the discretised system:

$$x_{k+1} = \underbrace{\left(I - \frac{\Delta t}{2} A\right)^{-1} \left(I + \frac{\Delta t}{2} A\right)}_{\bar{A}} x_k + \underbrace{\left(I - \frac{\Delta t}{2} A\right)^{-1} \frac{\Delta t}{2} B}_{\bar{B}} (u_k + u_{k+1}) \tag{23}$$

Note on Input Handling: In many sequence modelling implementations (e.g. S4), a Zero-Order Hold (ZOH) is assumed for the input ($u(\tau) \approx u_k$). In that specific case, the input term simplifies to $(I - \frac{\Delta t}{2} A)^{-1} \Delta t B u_k$.

3.2 Geometric Derivation via Gradient Approximation

An alternative derivation can be established by examining the geometric interpretation of the derivative. Rather than integrating the area under the curve, we approximate the state trajectory using the finite difference quotient.

Recall the definition of the derivative:

$$\dot{x}(t) = \lim_{\Delta t \rightarrow 0} \frac{x(t + \Delta t) - x(t)}{\Delta t} \tag{24}$$

For a finite step size Δt , we can approximate the next state x_{k+1} by projecting from the current state x_k along a gradient vector $\mathbf{k}$:

$$x_{k+1} \approx x_k + \Delta t \cdot \mathbf{k} \tag{25}$$

The continuous ODE gives us the exact instantaneous gradient at any point: $\dot{x}(t) = Ax(t) + Bu(t)$. The difference between the discretisation methods lies entirely in *at which point in time* we choose to sample this gradient $\mathbf{k}$.

3.2.1 Forward Euler (Left-Sided Slope)

Geometric Intuition: We assume the gradient remains constant throughout the interval $[t, t + \Delta t]$, fixed at its initial value. We simply extend the tangent line at $x(t)$ forward in time.

- **Gradient Choice:** $\mathbf{k} = \dot{x}(t) = Ax_k + Bu_k$

Substituting this into Eq. (25) yields the explicit update:

$$x_{k+1} = (I + \Delta t A)x_k + \Delta t Bu_k \quad (26)$$

3.2.2 Backward Euler (Right-Sided Slope)

Geometric Intuition: We assume the gradient across the interval is determined by the slope at the *destination* point $x(t + \Delta t)$. This is akin to finding a point in the future such that looking backwards with that slope leads to our current position.

- **Gradient Choice:** $\mathbf{k} = \dot{x}(t + \Delta t) = Ax_{k+1} + Bu_{k+1}$

Substituting this into the projection equation:

$$x_{k+1} = x_k + \Delta t(Ax_{k+1} + Bu_{k+1}) \quad (27)$$

Rearranging to solve for the unknown x_{k+1} :

$$\begin{aligned} x_{k+1} - \Delta t Ax_{k+1} &= x_k + \Delta t Bu_{k+1} \\ x_{k+1} &= (I - \Delta t A)^{-1} x_k + (I - \Delta t A)^{-1} \Delta t Bu_{k+1} \end{aligned} \quad (28)$$

3.2.3 Bilinear / Trapezoidal (Average Slope)

Geometric Intuition: To achieve a more accurate approximation, we define the gradient $\mathbf{k}$ as the arithmetic mean of the initial tangent (at t) and the final tangent (at $t + \Delta t$). This compensates for the curvature of the trajectory.

- **Gradient Choice:** $\mathbf{k} = \frac{1}{2} (\dot{x}(t) + \dot{x}(t + \Delta t))$

Substituting the ODE definitions for the slopes:

$$\mathbf{k} = \frac{1}{2} [(Ax_k + Bu_k) + (Ax_{k+1} + Bu_{k+1})] \quad (29)$$

Plugging this into the projection equation and grouping terms:

$$\begin{aligned} x_{k+1} &= x_k + \frac{\Delta t}{2} (Ax_k + Ax_{k+1}) + \frac{\Delta t}{2} (Bu_k + Bu_{k+1}) \\ \left(I - \frac{\Delta t}{2} A\right) x_{k+1} &= \left(I + \frac{\Delta t}{2} A\right) x_k + \frac{\Delta t}{2} B(u_k + u_{k+1}) \end{aligned} \quad (30)$$

Which results in the familiar Bilinear form:

$$x_{k+1} = \left(I - \frac{\Delta t}{2} A\right)^{-1} \left(I + \frac{\Delta t}{2} A\right) x_k + \dots \quad (31)$$

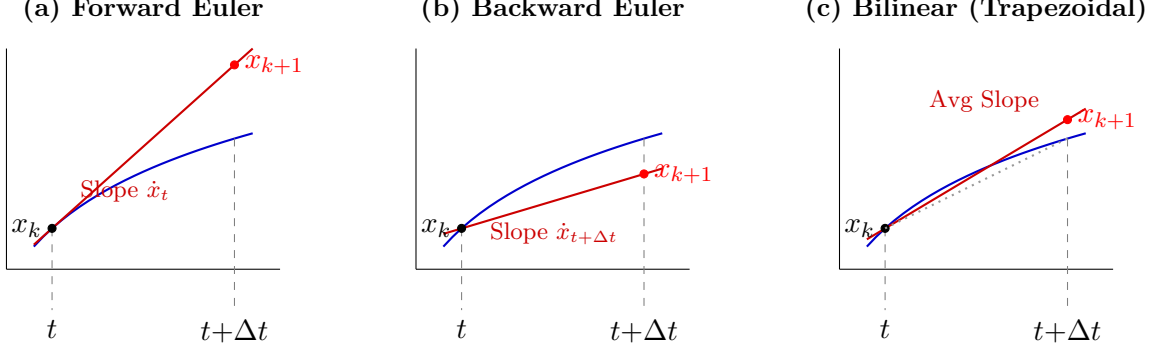


Figure 3: **Geometric Interpretation.** Comparison of gradient approximations. Forward Euler (Left) overshoots on concave curves; Backward Euler (Middle) heavily damps the step; Bilinear (Right) uses the average slope, closely tracking the curvature.

3.3 Comparative Analysis: Selecting the Optimal Discretisation

Having established three distinct discretisation methods (Forward Euler, Backward Euler, and Bilinear) via both integration and gradient approximation, a natural question arises: *Which method yields the most accurate representation of the underlying continuous dynamics?*

To answer this, we employ a two-fold evaluation strategy: first, an empirical reconstruction task to observe stability and phase properties; and second, a theoretical comparison against the analytical exact solution.

3.3.1 Empirical Validation: The Reconstruction Task

To complement the theoretical derivations, we evaluate the discretisation methods on a Memory Reconstruction task. This experiment tests the system’s ability to maintain a memory of the input signal over a long duration.

We define a State Space Model with state dimensions $N \in \{64, 128, 256\}$. The task is to reconstruct a stochastic input signal $u(t)$ after a fixed time delay.

- **Input Signal:** White Noise ($\mathcal{N}(0, 1)$), chosen for its high-frequency content which stresses the stability of the discretisation.
- **Lag:** 100 time steps.
- **Sequence Length:** $L = 2000$.
- **Step Size:** $\Delta t = 0.01$.

The reconstruction performance (R^2 score) for Forward Euler, Backward Euler, and Bilinear discretisation is visualised in Figure 4. Empirical results demonstrate that whilst Forward Euler diverges due to instability and Backward Euler suffers from severe artificial damping, the Bilinear transform uniquely preserves the system’s energy. This allows it to achieve perfect reconstruction ($R^2 = 1.0$) by maintaining stability without sacrificing high-frequency signal fidelity.

Although these empirical results favor the Bilinear method, visual inspection alone is insufficient. We require a mathematical ground truth to quantify the approximation error. We now turn to the derivation of the **Exact Solution** under the Zero-Order Hold (ZOH) assumption to benchmark each discretisation method.

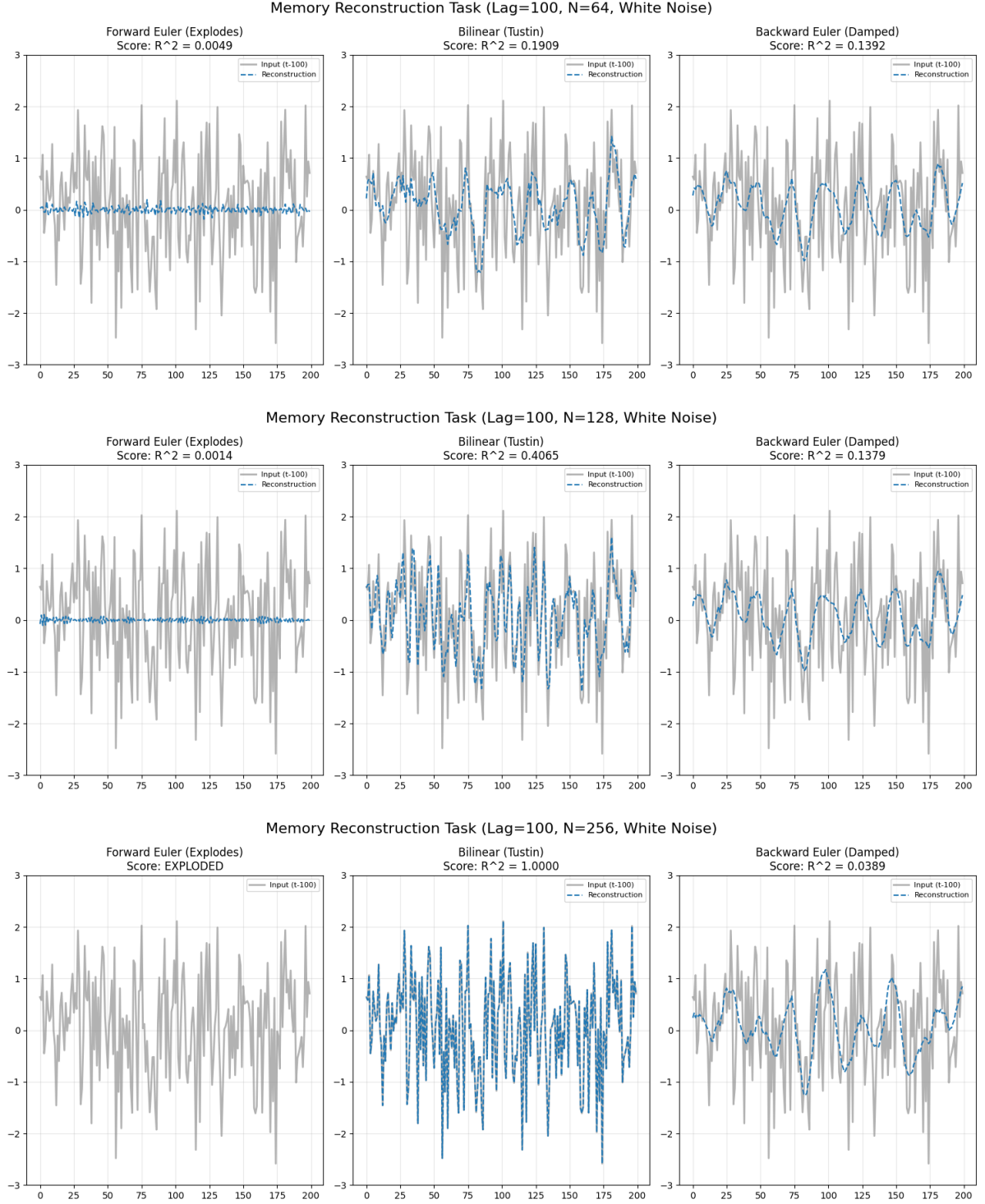


Figure 4: Memory Reconstruction Task. Comparison of discretisation methods across increasing state dimensions N . The blue dashed line represents the model's reconstruction of the grey input signal.

3.4 The Exact Solution (ZOH)

To mathematically benchmark the approximation methods, we derive the **Exact Solution** of the linear ODE. This serves as the third and final derivation path.

Recall the continuous ODE:

$$\dot{x}(t) = Ax(t) + Bu(t) \quad (32)$$

Using the integrating factor e^{-At} , we can solve this differential equation analytically over the interval $[t, t + \Delta t]$. We proceed by integrating both sides:

$$\frac{d}{dt}(e^{-At}x(t)) = e^{-At}Bu(t) \quad (33)$$

$$\begin{aligned} \int_t^{t+\Delta t} \frac{d}{d\tau}(e^{-A\tau}x(\tau)) d\tau &= \int_t^{t+\Delta t} e^{-A\tau}Bu(\tau) d\tau \\ e^{-A(t+\Delta t)}x(t+\Delta t) - e^{-At}x(t) &= \int_t^{t+\Delta t} e^{-A\tau}Bu(\tau) d\tau \end{aligned} \quad (34)$$

Multiplying by $e^{A(t+\Delta t)}$ isolates the future state:

$$x(t + \Delta t) = \underbrace{e^{A\Delta t}}_{\bar{A}} x(t) + \int_t^{t+\Delta t} e^{A(t+\Delta t-\tau)} Bu(\tau) d\tau \quad (35)$$

To solve the integral involving the input $u(\tau)$, we apply the **Zero-Order Hold (ZOH) assumption**: the input is piecewise constant between sample steps ($u(\tau) = u_k$ for $\tau \in [t, t + \Delta t]$). This allows us to factor Bu_k out of the integral. Defining $\alpha = t + \Delta t - \tau$, we obtain:

$$x_{k+1} = e^{A\Delta t} x_k + \left(\int_0^{\Delta t} e^{A\alpha} d\alpha \right) Bu_k \quad (36)$$

Thus, the **Exact discretisation matrix** is the matrix exponential:

$$\bar{A}_{\text{exact}} = e^{A\Delta t} \quad (37)$$

3.5 Comparisons via Taylor Expansion

To theoretically justify the preference for the Bilinear method (and its adoption in architectures such as LSSL), we compare the Taylor series expansions of the discretised state transition matrices $\bar{A}$.

3.5.1 Summary of Discretisation Matrices

First, we state the state transition matrix $\bar{A}$ derived from each method. For algebraic convenience, we define $z = A\Delta t$. The transition matrices are:

- **Exact Solution (ZOH):** $\bar{A}_{\text{exact}} = e^z$.
- **Forward Euler:** $\bar{A}_{\text{FE}} = I + z$.
- **Backward Euler:** $\bar{A}_{\text{BE}} = (I - z)^{-1}$.
- **Bilinear (Tustin):** $\bar{A}_{\text{Bil}} = (I - \frac{1}{2}z)^{-1} (I + \frac{1}{2}z)$.

3.5.2 Order of Error Analysis

We now expand these matrices with respect to z to observe their approximation fidelity relative to the Exact solution.

The Benchmark: Exact Solution The Taylor series for the matrix exponential serves as our ground truth:

$$\bar{A}_{\text{exact}} = e^z = I + z + \frac{1}{2}z^2 + \frac{1}{6}z^3 + \mathcal{O}(z^4) \quad (38)$$

Euler Methods (First-Order) Comparing the Euler approximations to Eq. (38):

$$\textbf{Forward Euler: } \bar{A}_{\text{FE}} = I + z \quad (39)$$

(Fails to capture $\frac{1}{2}z^2$. Error: $\mathcal{O}(\Delta t^2)$)

$$\textbf{Backward Euler: } \bar{A}_{\text{BE}} = (I - z)^{-1} = I + z + \mathbf{1}z^2 + \dots \quad (40)$$

(Coefficient mismatch: 1.0 vs 0.5 at z^2 causes damping)

Bilinear / Trapezoidal (Second-Order) We use the Neumann series expansion $(I - X)^{-1} \approx I + X + X^2$, setting $X = \frac{1}{2}z$:

$$\begin{aligned} \bar{A}_{\text{Bil}} &= \underbrace{\left(I + \frac{1}{2}z + \frac{1}{4}z^2 + \dots\right)}_{\text{Inverse Part}} \left(I + \frac{1}{2}z\right) \\ &= \left(I + \frac{1}{2}z\right) + \frac{1}{2}z \left(I + \frac{1}{2}z\right) + \frac{1}{4}z^2 \left(I + \frac{1}{2}z\right) + \mathcal{O}(z^3) \\ &= I + z + \left(\frac{1}{4} + \frac{1}{4}\right)z^2 + \mathcal{O}(z^3) \\ &= I + z + \frac{1}{2}z^2 + \mathcal{O}(z^3) \end{aligned} \quad (41)$$

Analysis: The Bilinear method matches the Exact expansion **exactly up to the second-order term** (z^2).

3.5.3 Conclusion

This derivation theoretically confirms that Bilinear discretisation is a **second-order accurate** method ($\mathcal{O}(\Delta t^3)$ local error), whereas Euler methods are only first-order accurate. This explains the superior signal reconstruction capabilities observed in the empirical tasks.

4 When God rolls the dice: ODE / RNN / CNN views

In the early 20th century, quantum mechanics forced physicists to grapple with a deeply uncomfortable reality. Albert Einstein famously declared that “God does not play dice,” expressing his profound unease with the irreducible randomness and seemingly contradictory nature of quantum theory. In response, Niels Bohr championed the principle of *complementarity*. He argued that light and matter can exhibit wave-like or particle-like properties depending entirely on how you measure them. To Bohr, these seemingly competing perspectives were not mutually exclusive; they were simultaneously valid and necessary for a complete understanding of reality.

Decades later, psychologist Edward de Bono introduced a parallel epistemological concept called the “Six Thinking Hats.” The core idea is that humans naturally argue over which perspective is the “correct” one. De Bono suggested instead that we should intentionally put on different “colored hats” to examine the exact same situation from entirely different, non-competing angles.

This framework applies directly to the architecture of State Space Models (SSMs). A modern SSM is governed by a single, elegant underlying mathematical structure. Yet, to truly understand how it operates in practice—and why it represents such a massive leap in sequence modeling—we must wear three different thinking hats.

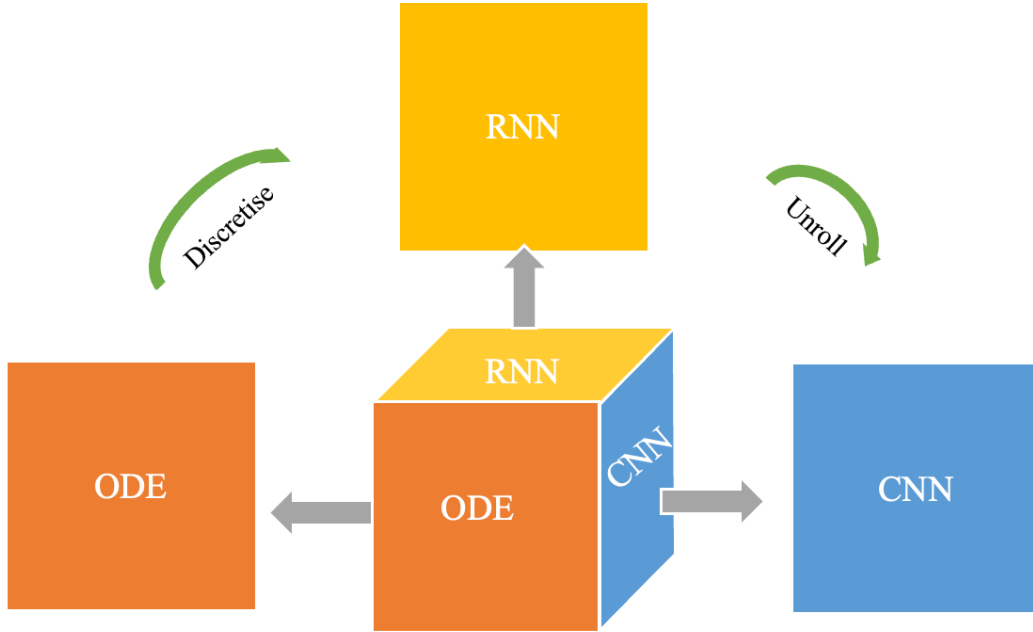


Figure 5

An SSM can be viewed through three distinct, complementary representations: continuous dynamics (ODE), sequential updates (RNN), and parallelizable kernels (CNN). We do not need to choose just one; we need all three.

Hat 1: The Continuous ODE View

Underneath everything, an SSM is a mathematical model of continuous time. If we put on the physicist’s hat, we see the world as smooth and uninterrupted.

As established in earlier chapters, the foundational representation of an SSM operates on continuous signals. It utilizes linear Ordinary Differential Equations (ODEs) to describe how the internal memory coefficients dynamically evolve over time.

The continuous-time system is defined as:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad (42)$$

$$y(t) = \mathbf{C}h(t). \quad (43)$$

This continuous ODE hat is vital because it explains *why* the model works fundamentally. It provides the rigorous mathematical framework for handling irregularly sampled data or signals operating at entirely unseen timescales.

Hat 2: The Discrete RNN View

However, modern computer hardware and digital data (like text tokens or digital audio) do not operate in continuous time. They are discrete.

When we put on the computer scientist's hat, we must discretize the continuous ODE using a step size Δ . Discretizing these continuous dynamics mathematically yields an efficient, closed-form recurrence. This gives us the Recurrent Neural Network (RNN) view.

$$h_k = \bar{\mathbf{A}}h_{k-1} + \bar{\mathbf{B}}x_k, \quad (44)$$

$$y_k = \mathbf{C}h_k. \quad (45)$$

In this view, the model processes data sequentially, one step at a time. This hat is highly prized for **inference** (generation). When a language model is generating text, it must predict the next word based on the previous one. An RNN is perfectly suited for this autoregressive task because its memory state h_k acts as a compact summary of all history, allowing for blazingly fast, constant-time updates during generation.

Hat 3: The Convolutional (CNN) View

If the RNN view is so efficient for inference, why do we need a third hat?

The problem with RNNs is **training**. If you want to train a model on a sequence of 100,000 tokens, an RNN must process step 1 to get to step 2, step 2 to get to step 3, and so on. It is an inherently sequential bottleneck that starves modern GPUs, which thrive on massive parallelization.

Here is where we put on the engineer's hat and perform a mathematical trick. Let's unroll the discrete RNN equations from a starting state of $h_{-1} = 0$.

At step 0:

$$h_0 = \bar{\mathbf{B}}x_0, \quad (46)$$

$$y_0 = \mathbf{C}\bar{\mathbf{B}}x_0. \quad (47)$$

At step 1:

$$h_1 = \bar{\mathbf{A}}h_0 + \bar{\mathbf{B}}x_1 = \bar{\mathbf{A}}(\bar{\mathbf{B}}x_0) + \bar{\mathbf{B}}x_1, \quad (48)$$

$$y_1 = \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}x_0 + \mathbf{C}\bar{\mathbf{B}}x_1. \quad (49)$$

At step 2:

$$h_2 = \bar{\mathbf{A}}h_1 + \bar{\mathbf{B}}x_2 \quad (50)$$

$$= \bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \bar{\mathbf{B}}x_2, \quad (51)$$

$$y_2 = \mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \mathbf{C}\bar{\mathbf{B}}x_2. \quad (52)$$

Notice a pattern? The output y_k is just a linear combination of the current and past inputs $(x_0, x_1, \dots, x_k)$. The coefficients multiplying these inputs are fixed: $\mathbf{C}\bar{\mathbf{B}}$, $\mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}$, $\mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}$, and so on.

This means we can represent the entire sequence of outputs y as a simple, one-dimensional **convolution** over the input sequence x :

$$y = x * \bar{\mathbf{K}}, \quad (53)$$

where the convolutional kernel $\bar{\mathbf{K}}$ is defined as:

$$\bar{\mathbf{K}} = (\mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^k\bar{\mathbf{B}}). \quad (54)$$

By computing this kernel $\bar{\mathbf{K}}$ upfront, we completely bypass the sequential bottleneck. We can compute the outputs for the entire sequence simultaneously using Fast Fourier Transforms (FFTs) on GPU hardware.

Synthesis: The Best of All Worlds

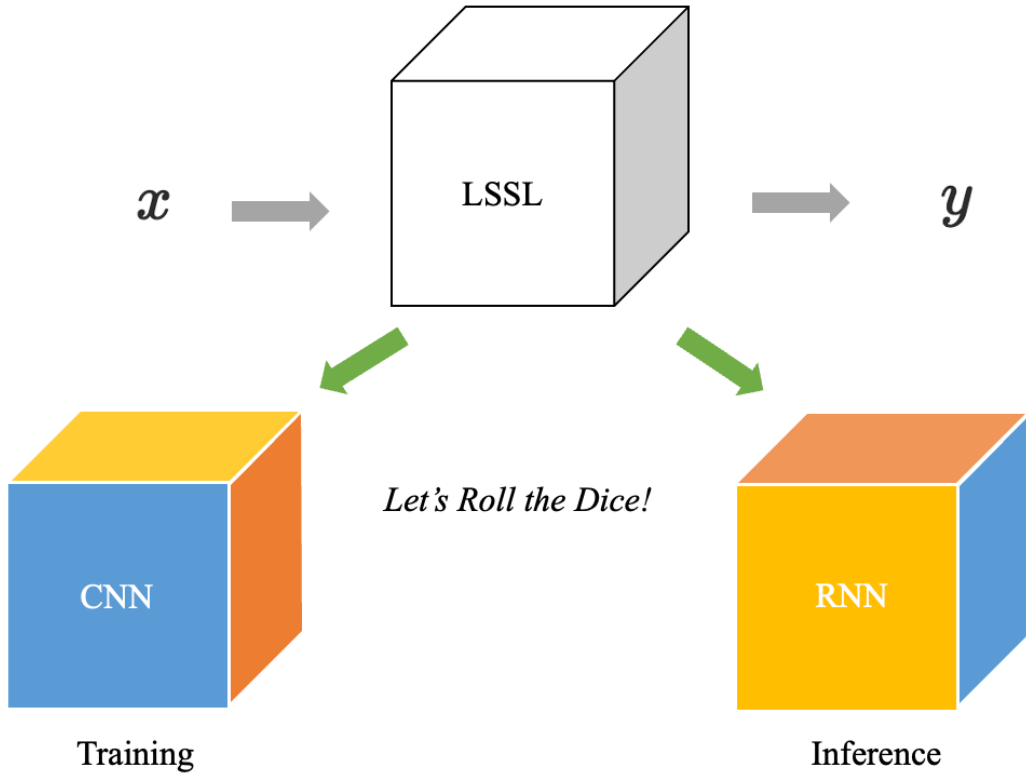


Figure 6

Pioneering works like the Linear State-Space Layer (LSSL) and the Structured State Space sequence model (S4) formalized this multi-faceted reality.

They realized that the underlying object—the State Space Model—is a single entity. However, by adopting Bohr’s complementarity, we unlock incredible engineering advantages. We use the **CNN view** during training to process millions of tokens in parallel, exploiting the full power of GPUs. Then, we seamlessly swap to the **RNN view** during deployment for lightning-fast, step-by-step inference. And the entire system remains mathematically grounded by the **ODE view**, ensuring the representations are optimal and robust.

5 Why Linear Time-Invariant (LTI) is not enough

Up to this point, we have built a beautiful, mathematically optimal memory machine using Linear Time-Invariant (LTI) State Space Models. But there is a glaring flaw in our design. By definition, an LTI system applies the exact same “forgetting” and “writing” rules to every single piece of data it encounters.

To understand why this is a catastrophic limitation for complex reasoning, we need to step outside of pure mathematics and look at how a truly intelligent system—the human brain—actually works.

5.1 The Neuroscience of Memory Budgeting

In cognitive neuroscience, memory formation is not “written” at a constant, robotic strength. A biological brain is a limited-capacity system that must constantly decide where to spend its representational budget.

When you experience a mundane, everyday event, your brain barely registers it. However, salient or stressful events trigger powerful neuromodulatory and hormonal signals—such as the release of noradrenaline and glucocorticoids. These chemicals act as a biological “override,” prioritizing specific experiences for intense consolidation while actively deemphasizing irrelevant background details. This is why emotionally charged memories remain unusually vivid decades later.

This selectivity can also be observed when the system breaks down. In Post-Traumatic Stress Disorder (PTSD), trauma-related memories become maladaptively over-strengthened and intrusive. Conversely, disruptions to the brain’s reconsolidation process can severely impair or weaken stored memories—a biological equivalent of “resetting” or “erasing” what was previously written.

An LTI model lacks this fundamental biological mechanism. It cannot flood its system with “hormones” to trigger a strong write, nor can it induce “amnesia” to reset its state. It blindly processes everything with the same fixed transition matrices.

5.2 The Information Theory of Surprisal

We can formalize this intuition using Information Theory. In our London weather story, imagine each day falls into one of K discrete “weather types”:

$$X \in \{1, 2, \dots, K\}. \quad (55)$$

Let the probability of each weather type occurring be $p_X(x)$. How much actual “information” do we gain from observing a particular outcome? In information theory, this is measured by *surprisal*:

$$\text{Surprisal} = -\log p_X(x). \quad (56)$$

This equation captures a profound truth: rare events carry exponentially more information than common ones.

For the everyday London drizzle, the probability $p_X(\text{drizzle})$ is large. Therefore, $-\log p_X(\text{drizzle})$ is a modest number. It tells you exactly what you already expected. But for an alien invasion, the probability $p_X(\text{alien})$ is infinitesimally small. Therefore, the surprisal $-\log p_X(\text{alien})$ is massive. Observing it delivers a staggering “information shock” to the system.

The average amount of information we expect to see in the stream is the Shannon entropy:

$$H(X) = -\sum_{x=1}^K p_X(x) \log p_X(x). \quad (57)$$

You can read this equation as a metric of the world’s unpredictability. In London, most days contribute only small-to-moderate surprisal, but the rare categories contribute massive surprisal when they finally happen.

5.3 The Need for Selectivity

This brings us to the ultimate downfall of the LTI State Space Model.

A sensible memory system should *never* write each day into memory with equal strength. Routine drizzle (low surprisal) should lead to a gentle, almost dismissive update. An alien invasion (high surprisal) must trigger a maximal write and a total memory reset, because it fundamentally changes what the future is likely to look like.

An LTI model, anchored by its fixed updating and forgetting profile, cannot naturally re-allocate its memory budget in response to these sharp spikes in information content. It treats the massive surprisal of an alien invasion with the exact same mathematical indifference as a cloudy Tuesday.

To move forward, we must abandon time-invariance. We need a model that can dynamically “read” the surprisal of its input and adapt its parameters on the fly. We need *Selectivity*.

6 Selective State Space: how Mamba makes B , C , and Δ_t time-varying

Since we have established that the rigid, “stubborn” nature of LTI (Linear Time-Invariant) systems cannot handle the violent fluctuations of real-world surprisal, the solution is clear: we need the memory write parameter B , the read parameter C , and the forgetting rate (time step) Δ_t to dynamically adapt to the current input x_t .

This sounds beautiful in theory, but how is it engineered in practice? If you open Mamba’s official GitHub repository (specifically `mamba.simple.py`), you will find that the authors use an incredibly elegant linear projection pipeline to mathematically “squeeze” these three dynamic parameters out of every single word—or every single day’s weather.

Let’s put on our engineer’s hat and deconstruct exactly how Mamba derives input-dependent Δ_t , B_t , and C_t through its forward pass.

Step 1: Expanding the Horizon (`in_proj`)

When day t ’s weather data x_t (with dimension d_{model}) arrives, Mamba’s first step is not to use it directly. Instead, an `in_proj` linear layer projects it into a higher-dimensional internal space, expanding the dimension to $2 \times d_{\text{inner}}$.

Once we have this high-dimensional vector, we slice it in half like a cake, resulting in two d_{inner} -dimensional branches: a main branch u_t and a gating branch z_t . The vector u_t will flow forward to generate our parameters, while z_t is held in reserve until the very end to modulate the final output. Expanding the input to a higher dimension to increase the feature capacity of the parameter generator is a classic deep learning technique for enhancing representational power.

Step 2: Looking Back at Yesterday (1D Causal Convolution)

Once we have our high-dimensional u_t , can we use it immediately to generate Δ_t ? No.

Think back to the alien invasion. If the model only looks at “today,” how can it know if an “alien arrival” is a bizarre anomaly or just something that happens every day? Without a sense of local history, the model cannot judge the surprisal of the current input.

Therefore, Mamba applies a depthwise causal 1D convolution to u_t , followed by a SiLU activation function, resulting in $\tilde{u}_t$ —a vector infused with local context. This step is absolutely critical; it provides a brief, local window into the past *before* any state update is computed.

Step 3: Slicing the Parameters (`x_proj`)

Now, $\tilde{u}_t$ possesses both high-definition details of today and an understanding of recent context. Next, we pass it through a linear projection layer called `x_proj`, mapping it into a vector of dimension $d_{\text{rank}} + 2 \times d_{\text{state}}$.

We slice this vector once more to directly extract the three components we desperately need:

1. B_t (dimension d_{state}): Today’s exclusive write matrix.
2. C_t (dimension d_{state}): Today’s exclusive read matrix.
3. Δ_t^{raw} (dimension d_{rank}): The “raw,” unrefined discretization step size.

You might ask: *Why are B_t and C_t in their final forms, while Δ_t is still “raw” and bottlenecked into a tiny d_{rank} dimension?* This is strictly for computational efficiency. Δ_t ultimately needs to broadcast across the d_{inner} -dimensional hidden state, but generating such a massive parameter directly is computationally exorbitant. Therefore, Mamba cleverly predicts Δ_t^{raw} in a small bottleneck first, and then projects it upward.

Step 4: The Alchemy of Step Size (`dt_proj` and Log-uniform Initialization)

Finally, we must refine Δ_t^{raw} into the final Δ_t .

First, the `dt_proj` layer projects Δ_t^{raw} back up to the d_{inner} dimension, yielding a vector s_t .

Next, to guarantee that the step size Δ_t is strictly positive in a physical sense (since time cannot flow backward), Mamba applies a `softplus` activation function:

$$\Delta_t = \text{softplus}(s_t + b_\Delta). \quad (58)$$

There is a hardcore engineering detail hidden here—the **initialization of the bias b_Δ** . At initialization, when the network weights are near zero, the generated step size Δ_t is roughly equal to $\text{softplus}(b_\Delta)$. To ensure the model starts its training equipped to handle both “long-term rote memorization” (small Δ) and “rapid responses to sudden events” (large Δ), the official code intentionally uses log-uniform sampling to initialize Δ_{init} , and then calculates the true b_Δ using the inverse `softplus` function. This meticulous initialization ensures that the time steps are evenly distributed across different orders of magnitude (e.g., 10^{-3} to 10^{-1}).

Evolution: From Mamba to Mamba-2’s Multi-Head View

At this point, we have successfully walked through the complete logic of `mamba_simple.py`. But this design still had room to evolve.

If you examine the recently released Mamba-2 (`mamba2.py`), you will notice a striking difference: Mamba-2 introduces a Transformer-like **Multi-Head mechanism**.

In early Mamba models, B_t and C_t were a single set of parameters for a given token within a batch. In Mamba-2, to further squeeze parallel computing performance out of GPUs and achieve what is known as *State Space Duality* (which we will explore deeply in later chapters), B_t , C_t , and even the state itself are divided into independent heads. This not only dramatically reduces parameter redundancy but also allows the underlying math of the SSM to achieve a beautiful, hardware-level unification with Multi-Head Attention.

Through this silky-smooth projection pipeline, Mamba successfully breathes “perception” into rigid mathematical formulas. When the input is mundane, it quietly retains; when aliens invade, it resets in an instant. This is the true power of the Selective State Space.

7 The Evolution of the SSM State Matrix A

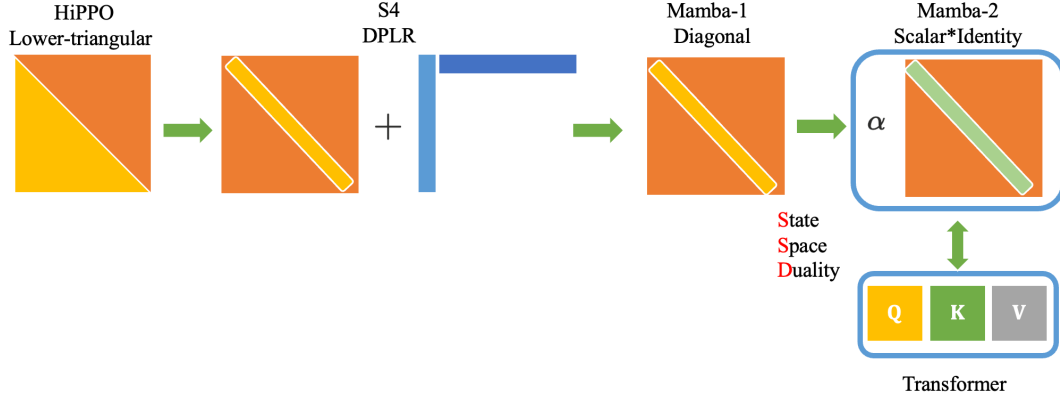


Figure 7: The evolution of state matrix A along the line of works

7.1 The general SSM form

A SSM (state space model) is basically a *Memory Machine* that:

- Keeps a hidden state h that acts like memory
- Updates that memory every step using a rule

A common (continuous-time) version SSM is typically written as:

$$\dot{h}(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t). \quad (59)$$

After discretisation with step size (Δ) , we obtain the discrete-time recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t. \quad (60)$$

Intuitively, state matrix A defines the internal memory dynamics. It determines **how past information is retained, mixed, and forgotten** in the latent state h .

If A is stable (roughly: eigenvalues with negative real parts), the system’s memory is well-behaved rather than exploding. Different structures for A (full, structured, diagonal, scalar) trade off **expressiveness vs. efficiency**.

Therefore, if we want to track the evolution of SSMs, tracking the evolution of matrix A is a clean way to see what changed and why. In this tutorial, we are going to focus on the evolution of matrix A following the line of works HiPPO–LSSL–S4–Mamba-01–Mamba-02.

7.2 How A evolve

Stage	What A ($\bar{A}$) is	Input dependence?
HiPPO (LegS)	A : Lower-triangular (Dense)	No (LTI)
LSSL	A : HiPPO-LegS, $\bar{A} = (I - \frac{\Delta}{2}A)^{-1} (I + \frac{\Delta}{2}A)$	No (LTI)
S4	DPLR parameterisation of A	No (LTI)
Mamba-01	A : Diagonal, $\bar{A}_t = \exp(\Delta_t A)$	Effective ($\bar{A}_t$) depends on (Δ_t)
Mamba-02	$\bar{A}_t = a_t I$	Effective ($\bar{A}_t$) depends on (Δ_t)

We’ll now walk through each stage, focusing solely on how A is chosen, parameterised, and used.

7.3 HiPPO: A as an optimal history compressor

HiPPO’s key idea is:

If you want to represent the entire history of a input signal $x(t)$ using a finite-dimensional state $h(t)$, you can do it by continuously projecting the input onto an **orthogonal polynomial basis**.

A possible choice of basis is from the **Legendre (scaled) family (LegS)**; the resulting A is derived from properties of Legendre polynomials. Intuitively:

- The system is designed so that $h(t)$ stores a **uniformly informative** summary of the entire past.
- It is constructed to be a global memory compressor.

Mathematically, we could write down the representation of state matrix $A \in \mathbb{R}^{N \times N}$ (using zero-based indices $n, k \in \{0, \dots, N-1\}$):

$$A_{nk} = - \begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k, \\ n+1 & \text{if } n = k, \\ 0 & \text{if } n < k. \end{cases} \quad (61)$$

A nice consequence is that the diagonal simplifies to

$$A_{nn} = -(n+1), \quad (62)$$

so the diagonal is $-(1, 2, \dots, N)$.

Here we consider a simple case $N = 4$; expanding the formula gives

$$A = - \begin{bmatrix} 1 & 0 & 0 & 0 \\ \sqrt{3} & 2 & 0 & 0 \\ \sqrt{5} & \sqrt{15} & 3 & 0 \\ \sqrt{7} & \sqrt{21} & \sqrt{35} & 4 \end{bmatrix}. \quad (63)$$

We could easily observe that at this stage, A is a **dense lower-triangular matrix**.

If you are interested how A is derived step by step, please refer to [under construction].

7.4 LSSL: HiPPO A + Bilinear discretisation

What changes from HiPPO?

Sequence modelling needs a discrete-time system. LSSL uses HiPPO-inspired state matrix A as an initialisation / inductive bias and then discretises.

A common choice here is **Bilinear (Tustin) discretisation**, which produces:

$$\bar{A} = \left(I - \frac{\Delta}{2}A\right)^{-1} \left(I + \frac{\Delta}{2}A\right). \quad (64)$$

If you are interested in why we chose Bilinear method, please refer to:

[appendix.pdf](#) (attachment)

At this stage, A is still conceptually **time-invariant** (LTI).

For LTI systems, we could interpret the system from both RNN and CNN views. Thus at this stage SSMs can be computed either way, ideally:

- Inferencing like an RNN, and
- Training like a CNN.

7.5 S4: Making A computational friendly with DPLR

Up to HiPPO/LSSL, we had A that compresses the entire history really well — but it’s **dense**, and doing anything like $(zI - A)^{-1}$ naively is expensive (people don’t normally like matrix inversion).

In S4, we want to keep the HiPPO-LegS structure, while rewriting A so the expensive linear algebra becomes cheap. That rewrite is **DPLR** (*Diagonal Plus Low-Rank*).

Why do we suddenly care about $(zI - A)^{-1}$?

S4 often computes an **SSM convolution kernel** rather than scanning an RNN state.

The kernel here is:

$$K_t = C\bar{A}^t\bar{B}. \quad (65)$$

Computing $\bar{A}^t$ directly is still expensive. Instead we jump to the z -transform:

$$\hat{K}(z) = \sum_{t \geq 0} K_t z^t = C(I - z\bar{A})^{-1}\bar{B}. \quad (66)$$

So the whole problem reduces to repeatedly evaluating something like a **resolvent**:

$$(I - z\bar{A})^{-1}. \quad (67)$$

Turn discrete resolvent into a continuous resolvent

S4 typically discretises in a way that lets us relate the discrete-time resolvent to a continuous-time one:

$$(I - z\bar{A})^{-1} \implies (\alpha(z)I - A)^{-1}. \quad (68)$$

Why this helps: $\alpha(z)$ changes with z , but A **stays the same**. So we need to evaluate $(\alpha(z)I - A)^{-1}$ for many z values efficiently (we write $(zI - A)^{-1}$ for simplicity).

DPLR structure: rewrite A as diagonal + low-rank

$$A = \Lambda + PQ^\top, \quad (\Lambda \text{ diagonal, } P, Q \in \mathbb{R}^{N \times r}). \quad (69)$$

Think of this as:

- Λ : the easy part (diagonal $\rightarrow$ cheap inverses)
- $PQ^\top$: a small low-rank correction that recovers the HiPPO structure

With DPLR, we can use **Woodbury**. Let

$$M = zI - A = (zI - \Lambda) - PQ^\top. \quad (70)$$

Define $D = zI - \Lambda$ (still diagonal). Woodbury says:

$$(D - PQ^\top)^{-1} = D^{-1} + D^{-1}P \left(I - Q^\top D^{-1}P \right)^{-1} Q^\top D^{-1}. \quad (71)$$

- D^{-1} is trivial (just invert diagonal entries).
- The remaining inverse is now on an $(r \times r)$ matrix:

$$\left(I - Q^\top D^{-1}P \right)^{-1}. \quad (72)$$

If $r = 1$, it’s a scalar.

So per z , the cost drops from $O(N^3)$ to roughly $O(N)$.

If you are interested in step-by-step derivation of DPLR and Woodbury, please refer to:

[appendix.pdf](#) (attachment)

7.5.1 From S4 to selective models: LTI becomes the bottleneck

At some point, the community realised a limitation: Pure LTI dynamics cannot *select* what to remember based on content.

From now on, we want to make Δ_t **input-dependent**, so the model can choose when to refresh memory and when to preserve it.

7.6 Mamba-01: Time dependent diagonal A + ZOH discretisation

The Mamba-01 paper describes two typical initialisations for the diagonal matrix A :

- **Real case:** $A_n = -(n + 1)$
- **Complex case:** $A_n = -\frac{1}{2} + ni$

An intuition is:

A is the system’s built-in *decay/oscillation rates*. It sets the time-scales of the latent state dimensions.

Let’s consider a simple example here. Taking $N = 4$ and using the usual **zero-based index** $n = 0, 1, 2, 3$:

Real case: $A_n = -(n + 1)$

Complex case: $A_n = -\frac{1}{2} + ni$

$$A_{\text{real}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -2 & 0 & 0 \\ 0 & 0 & -3 & 0 \\ 0 & 0 & 0 & -4 \end{bmatrix}, \quad A_{\text{complex}} = \begin{bmatrix} \frac{1}{2} & 0 & 0 & 0 \\ 0 & -\frac{1}{2} + i & 0 & 0 \\ 0 & 0 & -\frac{1}{2} + 2i & 0 \\ 0 & 0 & 0 & -\frac{1}{2} + 3i \end{bmatrix}. \quad (73)$$

The paper’s selective SSM makes Δ_t depend on the current token representation x_t , via

$$\Delta_t = \tau_{\Delta}(\text{bias} + s_{\Delta}(x_t)), \quad \tau_{\Delta} = \text{softplus}. \quad (74)$$

Δ_t is a per-token step size, so the model can decide when to update quickly and when to preserve memory.

- **If Δ_t is large:** the system takes a *big step* in time, which tends to **reset the current state** and **focus strongly on the current input**. Intuitively, the model is “selecting” this token and is more willing to forget what it was previously storing.
- **If Δ_t is small:** the system takes a *tiny step* in time, which tends to **preserve the current state** and **down-weight the current input**. Intuitively, the token is treated as more transient, and the model keeps its existing memory.

A helpful way to remember it is:

Large $\Delta_t \approx$ refresh memory from the current token. Small $\Delta_t \approx$ keep memory and let the token pass by.

Mamba-01 uses **ZOH discretisation**, where the discrete-time transition is

$$\bar{A}_t = \exp(\Delta_t A). \quad (75)$$

If you are interested in why we chose ZOH method, please refer to:

[appendix.pdf](#) (attachment)

Thus, even if A is time-invariant, $\bar{A}_t$ becomes **input-dependent** as Δ_t is input-dependent.

7.7 Mamba-02: Scalar Identity A as SSD

So far so good, right? Mamba-01 is almost proudly **non-Transformer**. It just shows that a selective SSM can stand on its own as a serious sequence layer.

However, reality check. Unfortunately, the entire ecosystem is **Transformer-shaped**. GPUs love big matmuls; software stacks love blocks that look like attention/MLP; and reviewers/users often want things that plug into that world cleanly.

And yes — there was a bit of drama. The original Mamba work (Mamba-01) was submitted to ICLR 2024, and the OpenReview thread shows a fairly intense review discussion.

Online discussion around that rejection also became a mini-meme, but let's not pretend social media is ground truth. If you want a quick break from the whole state matrix A thing, here are some clips of the discussion:

[D] Why is everybody surprised that Mamba got rejected from ICLR? Am I missing something?

Welcome back.

Mamba-02 is thus basically:

If the world insists on thinking in attention blocks, fine, let's meet it halfway.

*Let's rewrite SSMs so they **look like attention**, and also can be computed using the same sort of hardware-friendly primitives.*

Let's recall the famous equation for causal attention:

$$Y = (L \circ QK^\top)V. \quad (76)$$

What we are trying to answer is: *Can we choose an SSM parameterisation so that its sequence-to-sequence operator looks like masked attention?*

Recall: how a plain SSM becomes a causal matrix M

First we start from a discrete-time SSM:

$$h_t = A_t h_{t-1} + B_t x_t, \quad y_t = C_t^\top h_t. \quad (77)$$

If we unroll this recurrence, y_j can be written as a sum of contributions from all earlier inputs x_i (for $i \leq j$):

$$y_j = \sum_{i \leq j} \underbrace{C_j^\top A_j A_{j-1} \cdots A_{i+1} B_i}_{M_{ji}} x_i. \quad (78)$$

So the whole sequence map can be written as

$$y = Mx, \quad (79)$$

where M is **lower-triangular** and its entries are

$$M_{ji} = C_j^\top A_j A_{j-1} \cdots A_{i+1} B_i \quad (j \geq i). \quad (80)$$

This step matters because it turns *SSM as a recurrence* into *SSM as a matrix operator*, which is exactly what attention uses.

At this point, M is a $(T \times T)$ matrix. If we treat it as an arbitrary dense matrix, computing Mx is $O(T^2)$ and storing M is also $O(T^2)$. That's not efficient for long sequences.

So we need structure. This is where **semiseparable** matrices come in.

A semiseparable matrix has low-rank structure in every causal block.

That low-rank structure lets you multiply by M faster than $O(T^2)$, without explicitly forming M .

So far we have a general expression for M_{ji} . Now we look for a specific parameterisation of A_t that makes M factorise into an attention-ish expression.

The paper points out a particularly clean special case:

$$\boxed{A_t = a_t I \quad (\text{scalar} \times \text{identity})} \quad (81)$$

Then the product collapses:

$$A_j A_{j-1} \cdots A_{i+1} = (a_j a_{j-1} \cdots a_{i+1}) I = a_{j:i+1} I, \quad (82)$$

and therefore

$$\boxed{M_{ji} = a_{j:i+1} (C_j^\top B_i)} \quad (83)$$

This is the attention-shaped factorisation we are looking for:

- $(C_j^\top B_i)$ is a **content score** (a dot product: query–key style),
- $(a_{j:i+1})$ is a purely causal **mask/decay term** (depends only on positions between i and j).

Equivalently, you can think of

$$M \approx L_a \circ (CB^\top), \quad (84)$$

where L_a is a structured causal matrix built from the scalar chain $(a_{j:i+1})$, and $\circ$ is elementwise multiplication.

Causal attention is often written as:

$$Y = (L \circ QK^\top)V, \quad (85)$$

so the mapping is immediate:

- $(Q \leftrightarrow C)$
- $(K \leftrightarrow B)$
- $(V \leftrightarrow X)$
- $(L \leftrightarrow L_a)$

Remark. $A_t = a_t I$ is a special case, not the only possible route.

Choosing $A_t = a_t I$ is a particularly neat sufficient condition that makes the SSM operator look like masked attention.

It demonstrates **Existence**: there exists a structured SSM (within the semiseparable family) whose M is attention-shaped.

It does **not** claim uniqueness. There may be other parameterisations of A_t (or other structured matrix families) that also yield attention-like decompositions, or different fast algorithms. The paper focuses on scalar-identity as it gives a clean semiseparable form and strong efficiency properties.

8 Duality: why a SSM can look like a causal Transformer

In perception, cognition, and physics, we frequently encounter a profound phenomenon: a single underlying reality that requires two completely different, seemingly contradictory descriptions to be fully understood.

The most famous example is wave-particle duality in quantum mechanics. Are quantum entities like photons “really” waves, or are they “really” particles? In a naïve sense, they are neither, but our best understanding requires us to use both lenses, depending on what we are measuring and how we set up the experiment.

A second, more engineering-flavored example is time-frequency duality. The exact same audio signal can be understood as an evolving waveform over time, or as a static spectrum of frequencies. Crucially, due to the Convolution Theorem, a complex convolution operation in the time domain mathematically transforms into simple multiplication in the frequency domain. Changing our perspective does not change the signal; it simply makes the math dramatically easier to compute.

This philosophical and mathematical bridge brings us directly to the concept of **State Space Duality (SSD)**. And it is time we finally address the elephant in the room.

8.1 The Elephant in the Room: The Transformer

Since 2017, the Transformer architecture—specifically its core mechanism, Self-Attention—has dominated deep learning. Whenever Mamba or any other State Space Model is discussed, it is almost exclusively framed as a competitor: *SSM vs. Transformer. RNN vs. Attention*.

But what if this rivalry is an illusion? What if, just like a waveform and a frequency spectrum, they are simply two different lenses for viewing the exact same mathematical object?

State Space Duality (SSD) mathematically proves that linear state space models and structured masked attention mechanisms are, fundamentally, computing the exact same underlying semi-separable matrices. This realization shifts the narrative completely. The comparison stops being a superficial architectural battle and becomes a profound structural equivalence: they are simply two different ways to implement a causal mixing operator.

Let’s look at the math that bridges these two worlds.

8.2 The Mathematical Bridge: Mamba-2 and the Scalar Identity

To reveal how an SSM is actually a Transformer in disguise, the creators of Mamba-2 made a critical adjustment to the State Matrix A .

In previous chapters, we saw that making A a dynamic, diagonal matrix allowed for selective memory. In Mamba-2, to achieve State Space Duality (SSD) and unlock immense hardware efficiency, the structure of A is simplified into a *scalar identity*:

$$A_t = a_t I. \tag{86}$$

Here, a_t is simply a scalar value.

Why do this? Because it allows us to reframe the entire step-by-step SSM recurrence as a massive, single matrix operator: $Y = MX$.

If we unroll the recurrence and look at the interaction between a current time step j and a past time step i (where $j \geq i$), the mathematical relationship simplifies beautifully:

$$M_{ji} = (C_j^\top B_i) \cdot \prod_{k=i+1}^j a_k. \tag{87}$$

Now, let’s put on our “Transformer Hat” and look at this exact same equation.

In a standard causal linear attention mechanism, you have Queries (Q), Keys (K), and Values (V). Look at the structural mapping revealed in the equation above:

- C_j acts exactly as the **Query** (Q) at the current step j .
- B_i acts exactly as the **Key** (K) from the past step i .
- x_i (the input interacting with the matrix) acts exactly as the **Value** (V).

What about that trailing product, $\prod a_k$? In the Transformer view, this acts as a data-dependent decay mask, functionally identical to positional encoding or attention masking.

8.3 Duality

This is State Space Duality. Mamba-2 is an SSM, yet it computes a causal mixture of past information using Queries, Keys, and Values, just like a Transformer.

By proving this equivalence, the authors of Mamba-2 achieved something remarkable. Because Mamba-2 can be mathematically written as a form of attention, it can utilize the highly optimized, lightning-fast kernel implementations (like FlashAttention) that were originally built exclusively for Transformers.

We no longer have to choose between the sequential inference efficiency of an RNN and the highly parallelized training efficiency of Attention. Through the lens of SSD, they are one and the same. Mamba does not kill the Transformer; it absorbs it.

9 Mamba in engineering

